

Documentação do Projeto - Interpretador com Visitor Pattern

Este projeto implementa um interpretador de uma linguagem semelhante ao Lua, utilizando a técnica de análise sintática por meio de um parser LALR e a arquitetura baseada no padrão de projeto **Visitor**. O objetivo é fornecer uma estrutura robusta para percorrer e interpretar a árvore sintática abstrata (AST) gerada a partir do código-fonte.

Arquivo: parsetab.py

Este arquivo é **gerado automaticamente pelo PLY (Python Lex-Yacc)** e não deve ser editado manualmente.

Função:

Contém a **tabela de parsing** usada pelo analisador sintático. Define os estados, transições e produções válidas da gramática implementada no parser.

- `_lr_action`: dicionário de ações para o parser (shift/reduce).
- `_lr_goto`: transições de estados para não-terminais.
- `_lr_productions`: lista das produções da gramática.

Observações:

- O conteúdo é altamente técnico e baseado na gramática definida no parser (`ExpressionLanguageParser.py`).
- É essencial para o funcionamento interno do parser.

Arquivo: NodeClasses.py

Este arquivo contém a **representação da Árvore Sintática Abstrata (AST)**, que modela a estrutura do programa analisado. Cada classe representa um tipo de nó na árvore.

Organização:

1. **Classes Abstratas (ABCMeta):**
 - Representam categorias de elementos da linguagem como `Command`, `Program`, `Exp`, `Block`, etc.
 - Usam o método abstrato `accept(self, visitor)` para permitir a aplicação do Visitor Pattern.
2. **Classes Concretas:**
 - Ex: `CommandAtrib`, `CommandStructWhile`, `CommandIf`, etc.
 - Implementam o método `accept`, que chama o método correspondente no `Visitor`.
 - Armazenam atributos específicos de cada construção (ex: expressões, comandos internos, blocos).

3. Componentes Representados:

- **Programas e Blocos:** ProgramConcrete, BlockConcrete, etc.
- **Comandos:** atribuições, repetições (for, while, repeat), condicionais (if, else), funções, retornos, etc.
- **Expressões:** aritméticas, booleanas, prefixadas, chamadas de função, etc.
- **Listas e estruturas auxiliares:** ListVars, ListExps, ListNames, etc.

4. Funções e Argumentos:

- DefFuncao, ConcreteDefFunction, ConcreteFunction, Args, BodyFunction, etc.
- Permite definir funções com parâmetros e corpo de execução.

5. Controle de Fluxo:

- IfConcrete, StructWhile, StructRepeat, StructForIn, StructFor, etc.
- Representam blocos condicionais e de repetição.

6. Visitação e Estrutura:

- Cada classe implementa o método accept que delega a lógica de visitação para o Visitor.

Arquivo: Visitor.py

Este arquivo implementa a lógica principal do **Visitor Pattern**, permitindo percorrer os nós da AST e realizar ações correspondentes (como interpretação, geração de código, etc).

Principais características:

- Herda de AbstractVisitor.
- Implementa um método visitX para cada tipo de nó da AST.

Funcionalidade Geral:

- Ao visitar um nó, imprime (ou poderia interpretar) o conteúdo de maneira ordenada.
- Mantém controle de indentação com a variável global tab e a função auxiliar blank().

Exemplos:

- visitCommandStructWhile: imprime a estrutura de um while com seu corpo.
- visitCommandIf2: imprime um if com else.
- visitCommandExpPlus: avalia uma soma de expressões.
- visitCommandArgs2: reconhece chamadas de função sem argumentos.

Uso Real:

Esse Visitor pode ser adaptado para:

- Interpretar a linguagem.
- Gerar código intermediário.

- Realizar análise semântica.
-

Como Funciona o Projeto

1. O código-fonte da linguagem é fornecido.
 2. O parser lê e valida a sintaxe com base na gramática.
 3. É gerada uma AST contendo objetos como ProgramConcrete, CommandIf, ExpPlus, etc.
 4. A AST é percorrida pelo Visitor, que executa ou imprime os blocos de código.
-

Benefícios da Arquitetura

- **Separação de responsabilidades:** lógica da linguagem é separada da lógica de execução.
 - **Escalabilidade:** fácil adicionar novos comandos ou tipos de expressão.
 - **Reutilização de código:** o Visitor pode ser reutilizado para diversas finalidades.
 - **Leitura facilitada:** estrutura em árvore deixa o código interpretado mais claro e organizado.
-

Possíveis Melhorias Futuras

- Implementar um visitor para avaliação de expressões numéricas.
- Criar uma versão com geração de bytecode ou código Python.
- Adicionar tratamento de erros semânticos e de tipo.
- Criar testes unitários para validar os nós da AST.
- Criar interface gráfica para exibir visualmente a AST.